



机器学习与 Python 实践

第 2 次作业

共享单车小时租借数量预测

姓名：

专业： 强基班

学号：

教师：

学院： 信息科学与工程学院

2026 年 05 月 22 日

目录

1	前言及问题概述	2
2	相关基础知识及方法	2
2.1	相关原理	2
2.2	数据描述	3
2.3	算法实现	5
3	实验结果与可扩展内容	6
3.1	实验结果	6
3.2	参数调优及可扩展内容	8
4	讨论与结论	9
4.1	结果讨论	9
4.2	结论总结	10
5	心得体会与思考	10
5.1	个人心得	10
5.2	启发与思考	10
6	附录	11
	算法代码及其注释	11
	关键运行结果汇总	28

1 前言及问题概述

共享单车系统是城市短距离出行中的重要组成部分。用户可以在不同地点租借和归还自行车，系统会记录每一次租借发生的日期、小时、天气、温度、湿度、风速和用户类型等信息。由于骑行需求受到时间规律、工作日与节假日、天气条件以及季节变化等多种因素影响，若能够根据这些环境和时间特征预测每小时租借数量，就可以为车辆调度、站点补车和运营维护提供参考。

本次实验围绕华盛顿 2011–2012 年每小时共享单车租赁数据展开。预测目标为 `cnt`，即每小时租借自行车总数。实验流程与题目要求保持一致，依次完成数据预览、相关分析、直接建模、特征工程、特征编码、离散化、构造新特征、异常值检测和处理、数值特征标准化以及重新建模。模型评价采用 RMSE、MAE、 R^2 和 NRMSE，其中 RMSE 和 MAE 越小表示误差越低， R^2 越接近 1 表示模型解释能力越强。

需要特别注意的是，原始字段中的 `casual` 和 `registered` 分别表示未注册用户和注册用户租赁数量，两者相加正好等于目标变量 `cnt`。如果将这两个字段作为输入特征，会造成严重目标泄漏，使模型结果失去实际意义。因此本实验在建模阶段删除 `casual` 和 `registered`，仅在数据分析阶段保留它们用于说明相关性。

2 相关基础知识及方法

本章从回归预测原理、数据集字段含义和算法实现流程三个方面说明实验方法。报告正文延续报告 1 的排版结构，但实验内容依据报告 2 文件夹中的代码、运行结果和可视化图表重新整理。

2.1 相关原理

共享单车租借数量预测属于监督学习中的回归任务。设每小时样本的特征向量为 $x \in \mathbb{R}^d$ ，目标变量为 $y = \text{cnt}$ ，回归模型的目标是学习一个函数 $f(x)$ ，使预测值

$$\hat{y} = f(x) \quad (1)$$

尽可能接近真实租借数量。若采用线性模型，常见形式为

$$\hat{y} = w_0 + w_1x_1 + \cdots + w_dx_d. \quad (2)$$

线性模型具有解释性强、训练速度快等优点，但对非线性时间规律和复杂交互关系的表达能力有限。

本实验同时使用岭回归和随机森林回归进行对比。岭回归在线性回归基础上加入 L_2 正则化项，目标函数可写为

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \|\mathbf{w}\|_2^2. \quad (3)$$

其中 λ 用于控制正则化强度，能够在一定程度上降低多重共线性和过拟合风险。

随机森林回归由多棵决策树集成得到。每棵树在不同样本和特征子集上训练，最终预测结果通常取多棵树预测值的平均：

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f_t(x). \quad (4)$$

与单一线性模型相比，随机森林能够捕捉非线性关系和变量交互，因此更适合处理小时、天气、温度和工作日等因素共同影响的出行需求预测问题。

由于租借数量为非负且呈明显右偏分布，实验还对部分模型使用 `log1p` 目标变换，即训练时拟合 $\log(1 + y)$ ，预测后再通过 `expm1` 还原。该处理可以缓解大值样本对损失函数的影响，使模型更加关注整体相对误差。

2.2 数据描述

数据集文件为 `hour.csv`，共包含 17379 条小时级记录，字段数为 17。程序读取后新增 `datetime` 字段，将日期 `dteday` 和小时 `hr` 组合为完整时间戳，因此数据预览阶段共有 18 个字段。主要字段含义见表 1。

表 1: 共享单车小时租赁数据集主要字段说明

字段	类型	含义
<code>datetime</code>	时间字段	详细到小时的日期和时间戳
<code>season</code>	类别特征	季节，1 为春天，2 为夏天，3 为秋天，4 为冬天
<code>holiday</code>	二值特征	是否为法定假期
<code>workingday</code>	二值特征	是否为工作日，非周末且非公共假期
<code>weathersit</code>	类别特征	天气情况，数值越大通常表示天气越差
<code>temp</code>	连续特征	归一化温度
<code>atemp</code>	连续特征	归一化体感温度
<code>hum</code>	连续特征	归一化相对湿度
<code>windspeed</code>	连续特征	归一化风速
<code>casual</code>	统计字段	未注册用户租借数量，建模时删除
<code>registered</code>	统计字段	注册用户租借数量，建模时删除
<code>cnt</code>	目标变量	每小时租借自行车总数

从描述性统计可知，`cnt` 最小值为 1，最大值为 977，均值为 189.46，中位数为 142，标准差为 181.39，说明每小时租借数量波动较大且分布右偏。数据中所有字段缺失值数量均为 0，因此不需要额外进行缺失值填补。

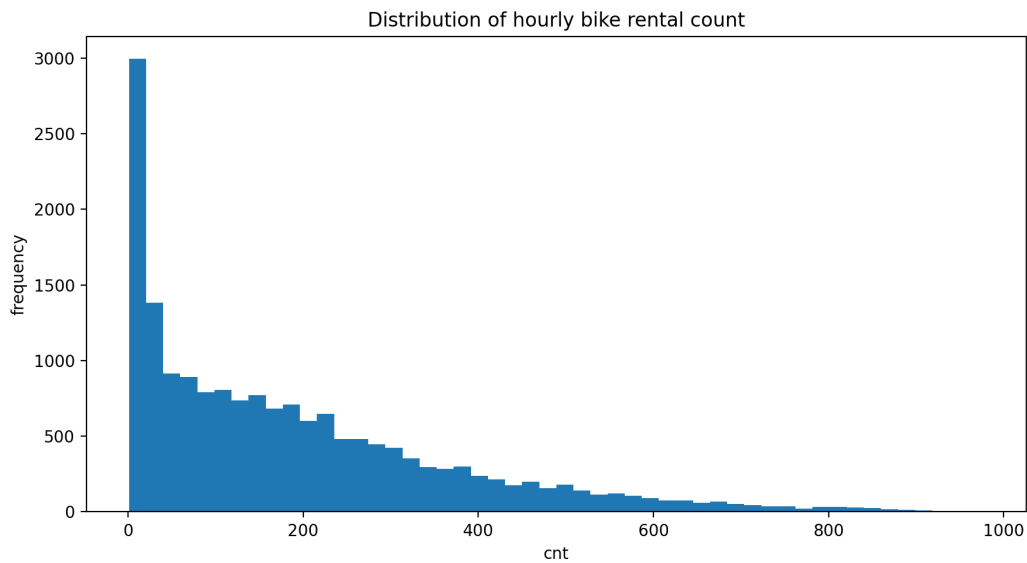


图 1: 每小时共享单车租借总数分布

图 1 显示，大量小时的租借数量集中在较低区间，而高租借量样本相对较少。这种长尾分布说明不同时间段的需求差异显著，也解释了为什么对目标变量进行对数变换可能有助于改善部分模型表现。

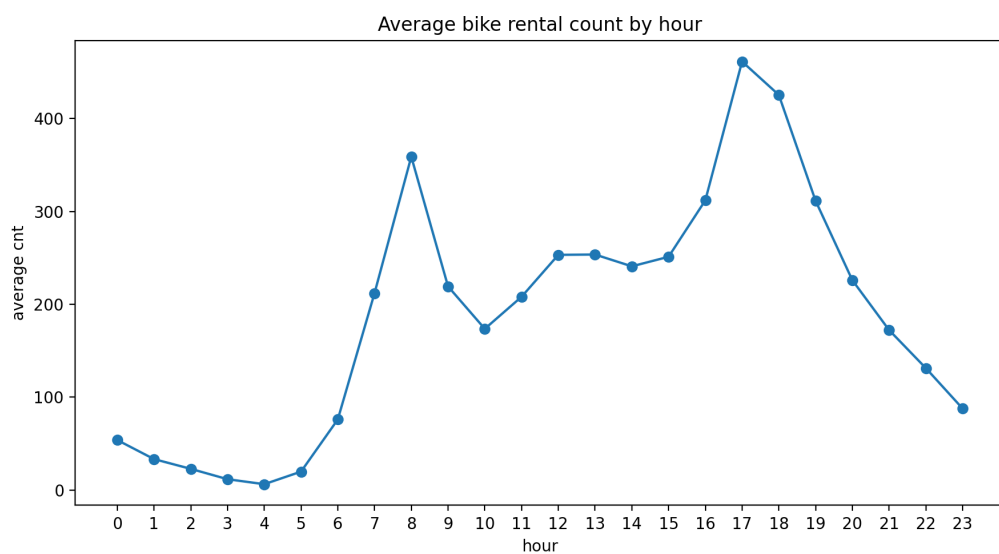


图 2: 不同小时的平均租借数量

从图 2 可以看到，租借数量具有明显小时周期规律。凌晨时段需求较低，白天逐步上升，并在通勤相关时段出现较高租借量。这说明小时特征是本任务中非常重要的预测变量。

2.3 算法实现

实验流程如下：

1. 使用 `pandas` 读取 `hour.csv`, 将 `dteday` 转换为日期格式, 并与 `hr` 合成 `datetime`。
2. 按时间顺序排序数据, 使用前 80% 样本作为训练集, 后 20% 样本作为测试集, 避免时间序列场景中的未来信息泄漏。
3. 进行数据预览, 保存前 10 行数据、数据类型、描述性统计、缺失值统计、目标变量分布图和每小时平均租借量图。
4. 计算数值字段相关系数, 分析各特征与 `cnt` 的关系, 并识别 `casual`、`registered` 这类目标泄漏字段。
5. 直接建模阶段删除 `cnt`、`datetime`、`dteday`、`instant`、`casual` 和 `registered`, 使用原始可用特征训练 `Ridge` 和 `RandomForest` 模型。
6. 特征工程阶段构造日期、周期、分箱、业务逻辑和交互特征, 包括 `hr_sin`、`hr_cos`、`is_night`、`is_peak_hour`、`temp_hum_interaction` 等。
7. 对类别特征进行 One-Hot 编码, 对连续数值特征进行 IQR 异常值裁剪和 `StandardScaler` 标准化, 对二值特征直接保留。
8. 重新训练特征工程后的 `Ridge` 和 `RandomForest` 模型, 输出模型评价指标、最佳模型预测曲线和随机森林特征重要性。

评价指标中, RMSE 定义为

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2}, \quad (5)$$

MAE 定义为

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}^{(i)} - y^{(i)}|, \quad (6)$$

决定系数 R^2 定义为

$$R^2 = 1 - \frac{\sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^n (y^{(i)} - \bar{y})^2}. \quad (7)$$

完整 Python 程序见附录。

3 实验结果与可扩展内容

3.1 实验结果

相关性分析结果表明，若包含统计字段，`registered` 与 `cnt` 的相关系数为 0.9722，`casual` 与 `cnt` 的相关系数为 0.6946。但二者是目标变量的组成部分，不能用于实际预测。去除泄漏字段后，与目标关系较明显的字段包括 `temp`、`atemp`、`hr` 和 `hum`，其中温度和小时与租借量正相关，湿度与租借量负相关。

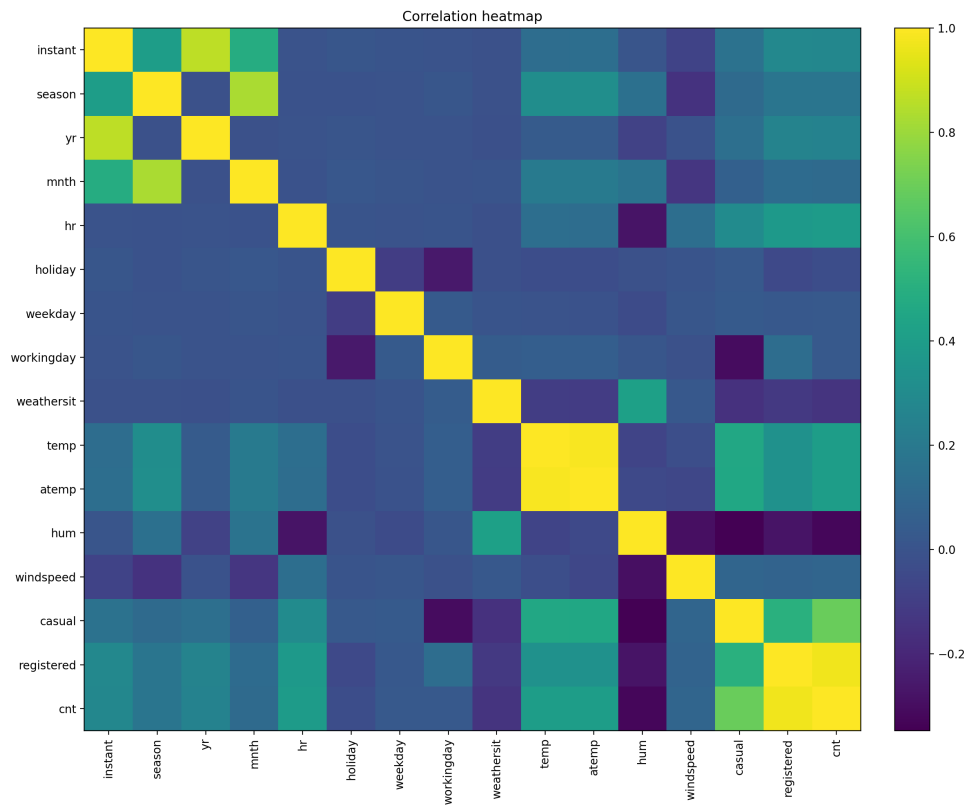


图 3: 数值字段相关性热力图

直接建模阶段使用原始可用字段进行训练，结果见表 2。在未进行复杂特征工程的情况下，随机森林已经取得较好效果，RMSE 为 69.90，MAE 为 45.02，测试集 R^2 为 0.8995；而 Ridge 线性模型即使使用目标对数变换， R^2 仅为 0.1520，说明原始编码下的线性模型难以充分刻画租借数量的非线性规律。

表 2: 直接建模阶段模型结果

模型	RMSE	MAE	R^2	NRMSE
Direct_RandomForest	69.90	45.02	0.8995	0.0716
Direct_Ridge_logTarget	203.04	139.83	0.1520	0.2080

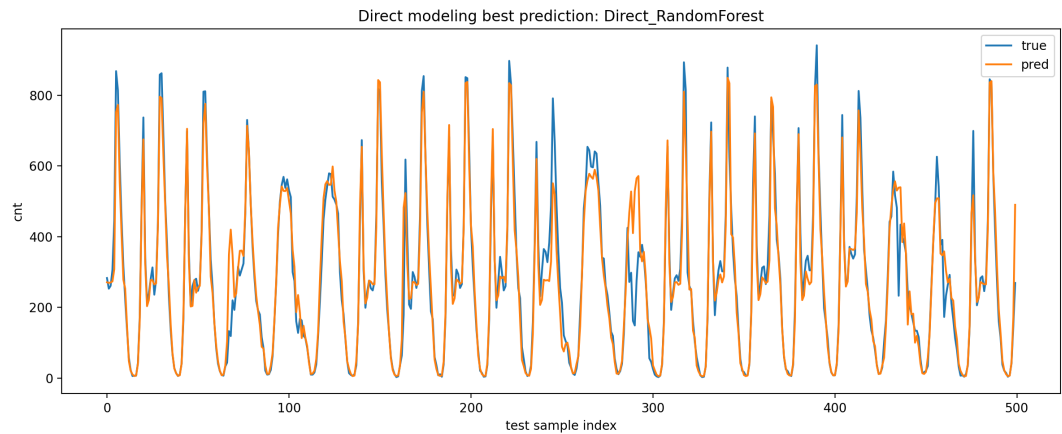


图 4: 直接建模最佳模型预测曲线

图 4 展示了直接建模阶段最佳模型在测试集前 500 个样本上的预测效果。整体趋势能够跟随真实租借数量变化，但在部分峰值处仍存在低估或滞后现象，这与共享单车需求的突发波动有关。

特征工程阶段构造了日期、周期、分箱、通勤、天气和交互特征，并对连续变量进行异常值处理。IQR 检测结果显示，原始连续气象变量中 `windspeed` 在训练集有 91 个异常值、测试集有 16 个异常值；构造特征中 `feel_temp_diff`、`temp_wind_interaction` 和 `hum_wind_interaction` 的异常值较多，因此使用训练集计算上下界并同时裁剪训练集和测试集，避免测试集信息泄漏。

表 3: 主要异常值裁剪结果

特征	下界	上界	训练异常数	测试异常数
<code>windspeed</code>	-0.1642	0.5523	91	16
<code>feel_temp_diff</code>	-0.0961	0.0567	392	61
<code>temp_wind_interaction</code>	-0.0921	0.2744	322	42
<code>hum_wind_interaction</code>	-0.0596	0.2809	426	77

经过特征工程、编码、离散化、异常值处理和标准化之后，重新建模结果见表 4。随机森林在特征工程后测试集 R^2 为 0.8774，仍保持较高水平；Ridge 模型提升非常明显， R^2 从直接建模阶段的 0.1520 提升至 0.8055，说明特征编码、周期特征和标准化对线性模型帮助很大。

表 4: 特征工程后重新建模结果

模型	RMSE	MAE	R^2	NRMSE
FE_RandomForest_logTarget	77.20	50.69	0.8774	0.0791
FE_Ridge_logTarget	97.23	61.15	0.8055	0.0996

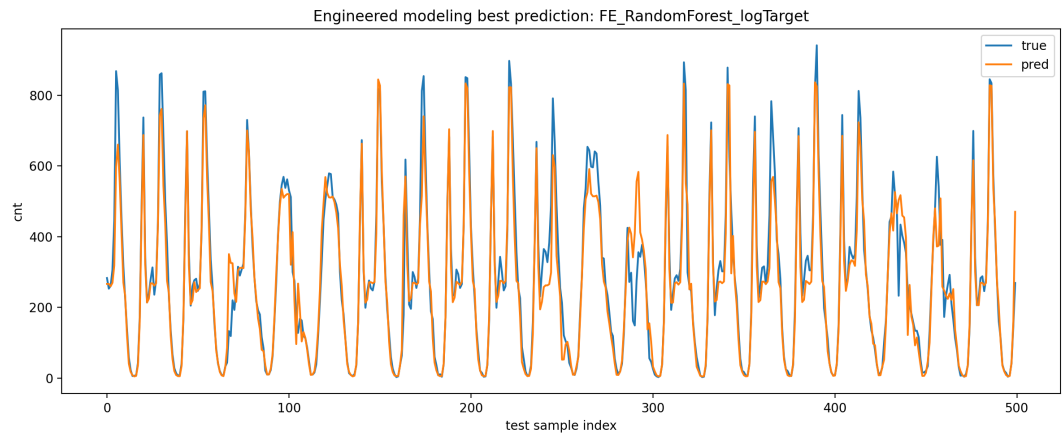


图 5: 特征工程后最佳模型预测曲线

将所有模型进行汇总，直接随机森林取得本次实验中最低 RMSE 和最高 R^2 ，是整体最优模型；特征工程后的随机森林略低于直接随机森林，可能与模型参数设置不同、目标对数变换以及特征处理方式有关。但从线性模型角度看，特征工程后的 Ridge 表现大幅改善，说明预处理流程本身是有效的。

表 5: 全部模型对比结果

模型	RMSE	MAE	R^2	阶段
Direct_RandomForest	69.90	45.02	0.8995	直接建模
FE_RandomForest_logTarget	77.20	50.69	0.8774	特征工程后
FE_Ridge_logTarget	97.23	61.15	0.8055	特征工程后
Direct_Ridge_logTarget	203.04	139.83	0.1520	直接建模

3.2 参数调优及可扩展内容

随机森林特征重要性结果见图 6。其中 is_night 重要性最高，为 0.4919；hr_sin 重要性为 0.1916；随后是 temp、is_peak_hour、atemp 和 hr_cos。这说明小时周期、夜间判断、通勤高峰和温度是预测每小时租借数量的关键因素。

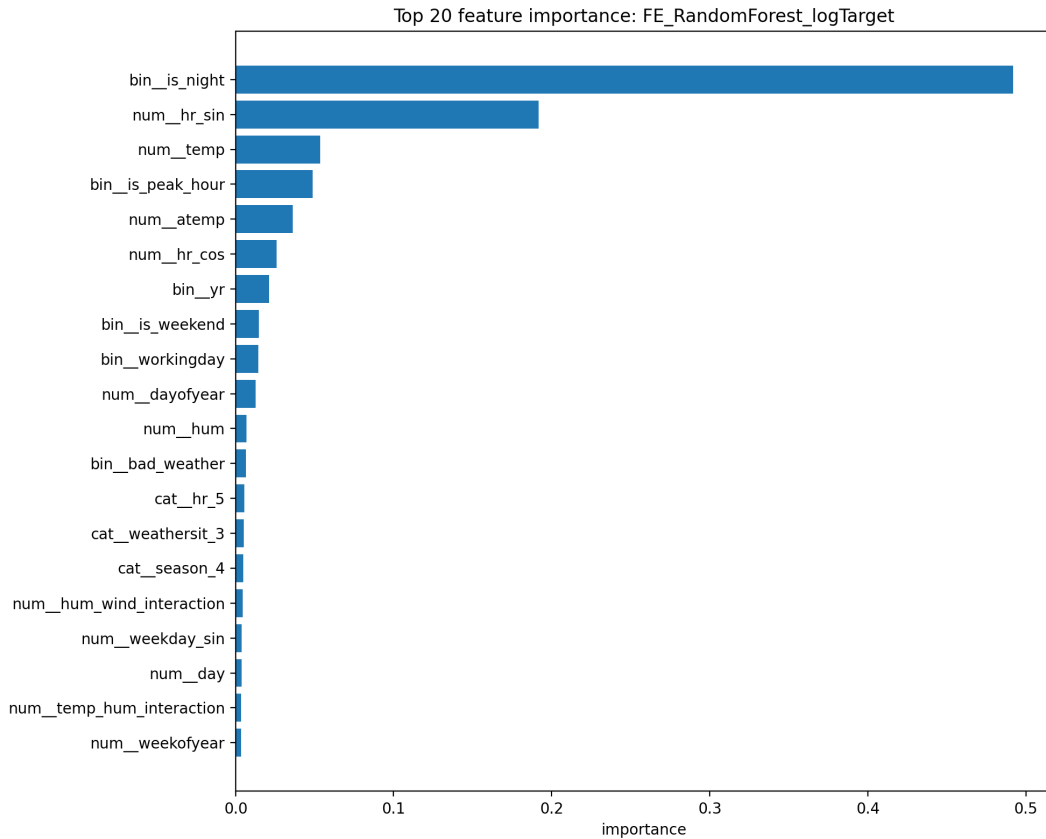


图 6: 特征工程后随机森林前 20 个重要特征

根据当前结果，后续可以从以下方向继续扩展。第一，可以对随机森林的 `n_estimators`、`max_depth`、`min_samples_leaf` 等参数进行网格搜索或随机搜索，寻找更稳定的泛化效果。第二，可以尝试 Gradient Boosting、XGBoost 或 LightGBM 等提升树模型，这类模型通常对表格型数据表现较好。第三，可以增加滚动时间窗口特征，例如过去 24 小时平均租借量、上一天同小时租借量等，以更充分利用时间序列信息。第四，可以分别预测 `casual` 与 `registered` 后再相加，从而刻画休闲用户和通勤用户不同的出行规律。

4 讨论与结论

4.1 结果讨论

通过实验可以得到以下结论。第一，共享单车小时租借数量具有明显时间周期性。凌晨需求最低，白天和通勤时段需求更高，因此小时特征及其周期编码是本任务中的核心信息。特征重要性中 `is_night`、`hr_sin` 和 `hr_cos` 排名靠前，也验证了这一点。

第二，目标泄漏字段必须严格排除。`casual` 和 `registered` 与 `cnt` 高度相关，但它们本身就是目标变量的组成部分。若将其作为输入，模型会得到虚假的高精度，无法

用于真实预测场景。本实验在建模阶段删除这两个字段，使评价结果更符合实际使用要求。

第三，树模型比简单线性模型更适合本任务。直接随机森林在原始可用特征上取得 $R^2 = 0.8995$ ，说明它能够自动学习非线性划分和变量交互。相比之下，直接 Ridge 模型表现较差，说明仅用原始数值编码的线性关系不足以描述租借需求。

第四，预处理和特征工程对线性模型提升明显。经过 One-Hot 编码、周期特征、离散化、业务特征、异常值裁剪和标准化之后，Ridge 的 R^2 提升到 0.8055。虽然该结果仍低于随机森林，但已经能较好解释租借数量变化，说明规范的数据预处理流程可以显著增强模型表达能力。

4.2 结论总结

本次实验完成了基于共享单车小时数据的租借数量预测任务。实验从数据预览和相关分析出发，先建立直接建模基线，再按照题目要求完成特征工程、特征编码、离散化、新特征构造、异常值处理、数值标准化和重新建模。最终结果显示，直接随机森林模型表现最好，RMSE 为 69.90，MAE 为 45.02， R^2 为 0.8995；特征工程后的 Ridge 模型相较直接 Ridge 有大幅提升，证明预处理过程对模型效果具有重要影响。

综合来看，共享单车需求预测不仅需要选择合适模型，也依赖于对数据含义的理解。时间、天气、工作日、通勤高峰和夜间等因素都与租借数量密切相关。通过本案例，可以较系统地理解回归预测任务中从数据预处理到模型评价的完整流程。

5 心得体会与思考

5.1 个人心得

通过本次实验，我对数据预处理在机器学习中的作用有了更直观的认识。模型训练并不是简单地把所有字段输入算法，而是需要先判断字段是否可用、是否存在泄漏、是否需要编码和标准化。例如 `casual` 与 `registered` 虽然和目标高度相关，但在真实预测中不能提前知道，因此必须删除。

同时，我也体会到不同模型对特征处理的依赖程度不同。随机森林能够较好处理非线性和交互关系，因此直接建模就能取得较好结果；而 Ridge 等线性模型更依赖合理的编码、标准化和特征构造。这个对比让我更清楚地理解了模型能力与特征工程之间的关系。

5.2 启发与思考

共享单车租借量受到时间、天气和用户出行习惯共同影响，是一个具有明显业务背景的预测问题。仅从相关系数或单个字段出发很难完全解释需求变化，必须结合工

作日、通勤时间、夜间、天气好坏和温湿度交互等因素综合建模。

后续如果继续改进，可以把问题进一步扩展为时间序列预测任务，引入滞后特征和滚动统计特征；也可以尝试更强的集成学习模型，并通过交叉验证或按月份滚动验证评估模型稳定性。这样不仅可以提升预测精度，也能更接近实际共享单车运营中的调度场景。

6 附录

算法代码及其注释

```
# -*- coding: utf-8 -*-
"""
共享单车小时租赁数量预测：数据预处理 + 特征工程 + 建模完整脚本

对应流程：
1. 数据预览
2. 相关分析
3. 直接建模
4. 特征工程
5. 特征编码
6. 离散化
7. 构造新特征
8. 异常值检测和处理
9. 数值特征标准化
10. 重新建模

运行方式：
    python bike_sharing_preprocessing_modeling_full_v1_20260522.py --data
    ↪ hour.csv

说明：
- 目标变量：cnt，即每小时自行车租借总数
- casual 和 registered 相加等于 cnt，属于目标泄漏字段，建模时必须删除
- 默认采用按时间顺序 80%/20% 划分训练集和测试集，更符合“小时租赁数量预测”的场景
"""

import argparse
import io
import os
import warnings
```

```
from pathlib import Path

import numpy as np
import pandas as pd

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")

# =====
# 0. 基本配置
# =====

TARGET_COL = "cnt"

# casual + registered = cnt, 不能作为输入特征, 否则会造成严重的数据泄漏
LEAKAGE_COLS = ["casual", "registered"]

REQUIRED_COLS = [
    "dteday", "season", "yr", "mnth", "hr", "holiday", "weekday",
    "workingday", "weathersit", "temp", "atemp", "hum", "windspeed", "cnt"
]

def make_one_hot_encoder():
    """ 兼容不同 sklearn 版本的 OneHotEncoder 写法。"""
    try:
        return OneHotEncoder(handle_unknown="ignore", sparse_output=False)
    except TypeError:
        return OneHotEncoder(handle_unknown="ignore", sparse=False)
```

```

def ensure_dir(path):
    path = Path(path)
    path.mkdir(parents=True, exist_ok=True)
    return path

def resolve_data_path(user_path):
    """
    优先使用命令行传入路径。
    如果未传入，则在当前目录自动搜索包含 Bike Sharing Hour 数据字段的 csv。
    """
    if user_path is not None:
        p = Path(user_path)
        if p.exists():
            return p
        raise FileNotFoundError(f" 找不到数据文件: {user_path}")

    candidates = list(Path(".").glob("*.csv"))
    for p in candidates:
        try:
            tmp = pd.read_csv(p, nrows=5)
            if all(c in tmp.columns for c in REQUIRED_COLS):
                return p
        except Exception:
            pass

    raise FileNotFoundError(
        " 未指定 --data, 且当前目录没有找到符合要求的 csv。 \n"
        " 请使用: python 脚本名.py --data hour.csv"
    )

# =====
# 1. 数据读取与预览
# =====

def load_data(data_path):
    df = pd.read_csv(data_path)

```

```
missing = [c for c in REQUIRED_COLS if c not in df.columns]
if missing:
    raise ValueError(f" 数据缺少必要字段: {missing}")

# 日期字段处理
df["dteday"] = pd.to_datetime(df["dteday"])
df["datetime"] = df["dteday"] + pd.to_timedelta(df["hr"], unit="h")

# 按时间排序, 保证后面时间切分合理
df = df.sort_values("datetime").reset_index(drop=True)
return df

def save_data_overview(df, out_dir):
    overview_dir = ensure_dir(out_dir / "01_data_overview")

    # 前 10 行
    df.head(10).to_csv(overview_dir / "head_10.csv", index=False,
        ↪ encoding="utf-8-sig")

    # info 信息
    buffer = io.StringIO()
    df.info(buf=buffer)
    (overview_dir / "data_info.txt").write_text(buffer.getvalue(),
        ↪ encoding="utf-8")

    # 描述性统计
    df.describe(include="all").to_csv(overview_dir / "describe.csv",
        ↪ encoding="utf-8-sig")

    # 缺失值统计
    missing_df = pd.DataFrame({
        "missing_count": df.isna().sum(),
        "missing_ratio": df.isna().mean()
    }).sort_values("missing_count", ascending=False)
    missing_df.to_csv(overview_dir / "missing_values.csv",
        ↪ encoding="utf-8-sig")

    # 目标变量分布图
    plt.figure(figsize=(9, 5))
    plt.hist(df[TARGET_COL], bins=50)
```

```

plt.title("Distribution of hourly bike rental count")
plt.xlabel("cnt")
plt.ylabel("frequency")
plt.tight_layout()
plt.savefig(overview_dir / "target_cnt_distribution.png", dpi=200)
plt.close()

# 每小时平均租借量
hourly_mean = df.groupby("hr")[TARGET_COL].mean()
plt.figure(figsize=(9, 5))
plt.plot(hourly_mean.index, hourly_mean.values, marker="o")
plt.title("Average bike rental count by hour")
plt.xlabel("hour")
plt.ylabel("average cnt")
plt.xticks(range(0, 24))
plt.tight_layout()
plt.savefig(overview_dir / "average_cnt_by_hour.png", dpi=200)
plt.close()

print("\n[1] 数据预览完成")
print(f"    数据规模: {df.shape[0]} 行, {df.shape[1]} 列")
print(f"    预览文件保存到: {overview_dir}")

# =====
# 2. 相关分析
# =====

def save_correlation_analysis(df, out_dir):
    corr_dir = ensure_dir(out_dir / "02_correlation_analysis")

    # 数值相关性, 保留 casual/registered 供说明, 但建模不会使用
    numeric_df = df.select_dtypes(include=[np.number]).copy()
    corr = numeric_df.corr()
    corr.to_csv(corr_dir / "correlation_matrix.csv", encoding="utf-8-sig")

    # 与目标 cnt 的相关性
    target_corr = corr[TARGET_COL].sort_values(ascending=False)
    target_corr.to_csv(corr_dir / "target_cnt_correlation.csv",
        ↪ encoding="utf-8-sig")

```



```

# 热力图
plt.figure(figsize=(12, 10))
im = plt.imshow(corr.values, aspect="auto")
plt.colorbar(im, fraction=0.046, pad=0.04)
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.index)), corr.index)
plt.title("Correlation heatmap")
plt.tight_layout()
plt.savefig(corr_dir / "correlation_heatmap.png", dpi=200)
plt.close()

print("\n[2] 相关分析完成")
print("    注意: casual 和 registered 与 cnt 高度相关,
    ↪ 但它们是目标泄漏字段, 建模时已删除。")
print(f"    相关性结果保存到: {corr_dir}")

# =====
# 3. 通用建模工具
# =====

def time_order_train_test_split(df, test_ratio=0.2):
    """
    时间序列类问题不建议随机打乱。
    这里用前 80% 时间作为训练集, 后 20% 时间作为测试集。
    """
    n = len(df)
    split_idx = int(n * (1 - test_ratio))
    train_df = df.iloc[:split_idx].copy()
    test_df = df.iloc[split_idx:].copy()
    return train_df, test_df

def regression_metrics(y_true, y_pred):
    y_pred = np.asarray(y_pred)
    y_pred = np.maximum(y_pred, 0) # 租赁数量不应为负

    mse = mean_squared_error(y_true, y_pred)
    rmse = float(np.sqrt(mse))
    mae = float(mean_absolute_error(y_true, y_pred))
    r2 = float(r2_score(y_true, y_pred))

```

```

y_true = np.asarray(y_true)
nrmse = rmse / (y_true.max() - y_true.min() + 1e-12)

return {
    "RMSE": rmse,
    "MAE": mae,
    "R2": r2,
    "NRMSE": float(nrmse)
}

def evaluate_model(model, X_train, y_train, X_test, y_test):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    return regression_metrics(y_test, pred), pred

def save_prediction_plot(y_true, y_pred, save_path, title, max_points=500):
    n = min(len(y_true), max_points)
    plt.figure(figsize=(12, 5))
    plt.plot(np.arange(n), np.asarray(y_true)[:n], label="true")
    plt.plot(np.arange(n), np.asarray(y_pred)[:n], label="pred")
    plt.title(title)
    plt.xlabel("test sample index")
    plt.ylabel("cnt")
    plt.legend()
    plt.tight_layout()
    plt.savefig(save_path, dpi=200)
    plt.close()

# =====
# 4. 直接建模：未做复杂预处理
# =====

def direct_modeling(df, out_dir):
    """
    直接建模：只删除明显不能直接输入/会泄漏的字段，然后直接训练模型。
    这一步作为基线，用于和后面的“预处理 + 特征工程”结果对比。
    """

```

```

model_dir = ensure_dir(out_dir / "03_direct_modeling")

drop_cols = [TARGET_COL, "datetime", "dteday", "instant"] + LEAKAGE_COLS
raw_features = [c for c in df.columns if c not in drop_cols]

train_df, test_df = time_order_train_test_split(df, test_ratio=0.2)

X_train = train_df[raw_features]
y_train = train_df[TARGET_COL]
X_test = test_df[raw_features]
y_test = test_df[TARGET_COL]

# 所有字段此时都是数值字段，但 season/weathersit 等仍只是原始编号
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), raw_features)
    ],
    remainder="drop"
)

models = {
    "Direct_Ridge_logTarget": TransformedTargetRegressor(
        regressor=Pipeline([
            ("preprocess", preprocessor),
            ("model", Ridge(alpha=1.0))
        ]),
        func=np.log1p,
        inverse_func=np.expm1
    ),
    "Direct_RandomForest": RandomForestRegressor(
        n_estimators=80,
        max_depth=18,
        min_samples_leaf=2,
        random_state=42,
        n_jobs=-1
    )
}

results = []
predictions = {}

```

```

# RandomForest 不需要标准化, 这里为了和 Ridge 处理流程统一,
↪ 仍保留原始数值输入
for name, model in models.items():
    if name == "Direct_RandomForest":
        model_to_fit = model
    else:
        model_to_fit = model

    metrics, pred = evaluate_model(model_to_fit, X_train, y_train,
    ↪ X_test, y_test)
    metrics["model"] = name
    results.append(metrics)
    predictions[name] = pred

result_df = pd.DataFrame(results).set_index("model").sort_values("RMSE")
result_df.to_csv(model_dir / "direct_model_comparison.csv",
↪ encoding="utf-8-sig")

best_name = result_df.index[0]
save_prediction_plot(
    y_test,
    predictions[best_name],
    model_dir / "direct_best_prediction_curve.png",
    f"Direct modeling best prediction: {best_name}"
)

print("\n[3] 直接建模完成")
print(result_df)
print(f"    直接建模结果保存到: {model_dir}")

return result_df

# =====
# 5-8. 特征工程、编码、离散化、新特征、异常值处理
# =====

def add_engineered_features(df):
    """
    特征工程:
    - 日期时间特征

```

```
- 周期性特征
- 离散化特征
- 业务逻辑新特征
"""
df = df.copy()

if "datetime" not in df.columns:
    df["dteday"] = pd.to_datetime(df["dteday"])
    df["datetime"] = df["dteday"] + pd.to_timedelta(df["hr"], unit="h")

# 日期特征
df["day"] = df["dteday"].dt.day
df["dayofyear"] = df["dteday"].dt.dayofyear
df["weekofyear"] = df["dteday"].dt.isocalendar().week.astype(int)

# 周期性特征：小时、月份、星期
df["hr_sin"] = np.sin(2 * np.pi * df["hr"] / 24)
df["hr_cos"] = np.cos(2 * np.pi * df["hr"] / 24)
df["mnth_sin"] = np.sin(2 * np.pi * df["mnth"] / 12)
df["mnth_cos"] = np.cos(2 * np.pi * df["mnth"] / 12)
df["weekday_sin"] = np.sin(2 * np.pi * df["weekday"] / 7)
df["weekday_cos"] = np.cos(2 * np.pi * df["weekday"] / 7)

# 离散化：把连续气象变量分箱
df["temp_bin"] = pd.cut(
    df["temp"],
    bins=[-np.inf, 0.25, 0.50, 0.75, np.inf],
    labels=["cold", "cool", "warm", "hot"]
)

df["hum_bin"] = pd.cut(
    df["hum"],
    bins=[-np.inf, 0.30, 0.60, 0.80, np.inf],
    labels=["dry", "normal", "wet", "very_wet"]
)

df["windspeed_bin"] = pd.cut(
    df["windspeed"],
    bins=[-np.inf, 0.15, 0.30, 0.45, np.inf],
    labels=["low", "middle", "high", "very_high"]
)
```

```

# 构造业务特征
df["is_weekend"] = df["weekday"].isin([0, 6]).astype(int)
df["is_night"] = ((df["hr"] <= 5) | (df["hr"] >= 22)).astype(int)
df["is_daytime"] = ((df["hr"] >= 7) & (df["hr"] <= 20)).astype(int)
df["is_commute_hour"] = df["hr"].isin([7, 8, 17, 18]).astype(int)

# 工作日通勤高峰、非工作日白天休闲高峰
df["is_peak_hour"] = (
    ((df["workingday"] == 1) & (df["hr"].isin([7, 8, 17, 18]))) |
    ((df["workingday"] == 0) & (df["hr"].between(10, 16)))
).astype(int)

df["bad_weather"] = (df["weathersit"] >= 3).astype(int)
df["good_weather"] = (df["weathersit"] == 1).astype(int)

# 交互特征
df["feel_temp_diff"] = df["atemp"] - df["temp"]
df["temp_hum_interaction"] = df["temp"] * df["hum"]
df["temp_wind_interaction"] = df["temp"] * df["windspeed"]
df["hum_wind_interaction"] = df["hum"] * df["windspeed"]
df["workingday_commute"] = df["workingday"] * df["is_commute_hour"]
df["weekend_daytime"] = df["is_weekend"] * df["is_daytime"]
df["bad_weather_peak"] = df["bad_weather"] * df["is_peak_hour"]

return df

def clip_outliers_iqr(train_df, test_df, numeric_cols, out_dir, k=1.5):
    """
    异常值处理：
    只用训练集计算 IQR 上下界，再同时裁剪训练集和测试集。
    这样可以避免使用测试集信息造成数据泄漏。
    """
    outlier_dir = ensure_dir(out_dir / "08_outlier_processing")

    train_df = train_df.copy()
    test_df = test_df.copy()

    records = []
    for col in numeric_cols:

```

```

q1 = train_df[col].quantile(0.25)
q3 = train_df[col].quantile(0.75)
iqr = q3 - q1

# 如果变量几乎没有波动，就跳过
if pd.isna(iqr) or iqr == 0:
    continue

low = q1 - k * iqr
high = q3 + k * iqr

train_outliers = ((train_df[col] < low) | (train_df[col] >
↪ high)).sum()
test_outliers = ((test_df[col] < low) | (test_df[col] > high)).sum()

train_df[col] = train_df[col].clip(low, high)
test_df[col] = test_df[col].clip(low, high)

records.append({
    "feature": col,
    "lower_bound": low,
    "upper_bound": high,
    "train_outlier_count": int(train_outliers),
    "test_outlier_count": int(test_outliers)
})

outlier_report = pd.DataFrame(records)
outlier_report.to_csv(outlier_dir / "iqr_outlier_clip_report.csv",
↪ index=False, encoding="utf-8-sig")

print("\n[8] 异常值检测和处理完成")
print(f"    IQR 裁剪报告保存到: {outlier_dir}")

return train_df, test_df, outlier_report

# =====
# 9-10. 标准化后重新建模
# =====

def engineered_modeling(df, out_dir):

```

```
model_dir = ensure_dir(out_dir / "10_engineered_modeling")

df_fe = add_engineered_features(df)

train_df, test_df = time_order_train_test_split(df_fe, test_ratio=0.2)

# 类别特征: 需要 one-hot 编码
categorical_features = [
    "season", "mnth", "hr", "weekday", "weathersit",
    "temp_bin", "hum_bin", "windspeed_bin"
]

# 二值特征: 可以直接作为数值输入
binary_features = [
    "yr", "holiday", "workingday",
    "is_weekend", "is_night", "is_daytime", "is_commute_hour",
    "is_peak_hour", "bad_weather", "good_weather",
    "workingday_commute", "weekend_daytime", "bad_weather_peak"
]

# 连续数值特征: 需要标准化
numeric_features = [
    "temp", "atemp", "hum", "windspeed",
    "day", "dayofyear", "weekofyear",
    "hr_sin", "hr_cos", "mnth_sin", "mnth_cos",
    "weekday_sin", "weekday_cos",
    "feel_temp_diff",
    "temp_hum_interaction", "temp_wind_interaction",
    ↪ "hum_wind_interaction"
]

# 异常值处理只对连续数值特征做, 不对类别编号和二值变量做
train_df, test_df, _ = clip_outliers_iqr(
    train_df=train_df,
    test_df=test_df,
    numeric_cols=numeric_features,
    out_dir=out_dir,
    k=1.5
)

feature_cols = categorical_features + binary_features + numeric_features
```



```
X_train = train_df[feature_cols]
y_train = train_df[TARGET_COL]
X_test = test_df[feature_cols]
y_test = test_df[TARGET_COL]

# 特征编码 + 数值标准化
# 类别特征: OneHotEncoder
# 连续特征: StandardScaler
# 二值特征: passthrough
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", make_one_hot_encoder(), categorical_features),
        ("num", StandardScaler(), numeric_features),
        ("bin", "passthrough", binary_features)
    ],
    remainder="drop"
)

models = {
    "FE_Ridge_logTarget": TransformedTargetRegressor(
        regressor=Pipeline([
            ("preprocess", preprocessor),
            ("model", Ridge(alpha=1.0))
        ]),
        func=np.log1p,
        inverse_func=np.expm1
    ),
    "FE_RandomForest_logTarget": TransformedTargetRegressor(
        regressor=Pipeline([
            ("preprocess", preprocessor),
            ("model", RandomForestRegressor(
                n_estimators=20,
                max_depth=14,
                min_samples_leaf=3,
                random_state=42,
                n_jobs=1
            ))
        ]),
        func=np.log1p,
        inverse_func=np.expm1
    )
}
```

```

    ),
}

results = []
predictions = {}
fitted_models = {}

for name, model in models.items():
    metrics, pred = evaluate_model(model, X_train, y_train, X_test,
    ↪ y_test)
    metrics["model"] = name
    results.append(metrics)
    predictions[name] = pred
    fitted_models[name] = model

result_df = pd.DataFrame(results).set_index("model").sort_values("RMSE")
result_df.to_csv(model_dir / "engineered_model_comparison.csv",
    ↪ encoding="utf-8-sig")

best_name = result_df.index[0]
best_pred = predictions[best_name]

# 保存最优模型预测曲线
save_prediction_plot(
    y_test,
    best_pred,
    model_dir / "engineered_best_prediction_curve.png",
    f"Engineered modeling best prediction: {best_name}",
    max_points=500
)

# 保存预测明细
pred_df = pd.DataFrame({
    "datetime": test_df["datetime"].values,
    "true_cnt": y_test.values,
    "pred_cnt": np.maximum(best_pred, 0)
})
pred_df.to_csv(model_dir / "best_model_predictions.csv", index=False,
    ↪ encoding="utf-8-sig")

# 尝试输出特征重要性：仅对树模型有效

```

```

try:
    best_model = fitted_models[best_name]
    inner_pipeline = best_model.regressor_
    final_model = inner_pipeline.named_steps["model"]
    if hasattr(final_model, "feature_importances_"):
        feature_names = inner_pipeline.named_steps["preprocess"].get_feature_names_out()
        importance_df = pd.DataFrame({
            "feature": feature_names,
            "importance": final_model.feature_importances_
        }).sort_values("importance", ascending=False)
        importance_df.to_csv(model_dir / "feature_importance.csv",
                               index=False, encoding="utf-8-sig")

        top = importance_df.head(20).iloc[::-1]
        plt.figure(figsize=(10, 8))
        plt.barh(top["feature"], top["importance"])
        plt.title(f"Top 20 feature importance: {best_name}")
        plt.xlabel("importance")
        plt.tight_layout()
        plt.savefig(model_dir / "feature_importance_top20.png", dpi=200)
        plt.close()
except Exception as e:
    print(f"    特征重要性输出跳过, 原因: {e}")

print("\n[4-10] 特征工程、编码、离散化、异常值处理、标准化和重新建模完成")
print(result_df)
print(f"    重新建模结果保存到: {model_dir}")

return result_df

# =====
# 主函数
# =====

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--data",
        type=str,

```

```
        default=None,
        help=" 共享单车小时数据 csv 路径, 例如 hour.csv"
    )
    parser.add_argument(
        "--out",
        type=str,
        default="results_bike_sharing_preprocessing",
        help=" 结果输出目录"
    )
    args = parser.parse_args()

    data_path = resolve_data_path(args.data)
    out_dir = ensure_dir(args.out)

    print("=" * 80)
    print(" 共享单车小时租赁数量预测: 数据预处理完整流程")
    print("=" * 80)
    print(f" 数据文件: {data_path}")
    print(f" 输出目录: {out_dir.resolve()}")

    df = load_data(data_path)

    # 1. 数据预览
    save_data_overview(df, out_dir)

    # 2. 相关分析
    save_correlation_analysis(df, out_dir)

    # 3. 直接建模
    direct_result = direct_modeling(df, out_dir)

    # 4-10. 特征工程后重新建模
    engineered_result = engineered_modeling(df, out_dir)

    # 汇总直接建模与重新建模结果
    summary_dir = ensure_dir(out_dir / "summary")
    direct_result2 = direct_result.copy()
    direct_result2["stage"] = "direct_modeling"

    engineered_result2 = engineered_result.copy()
    engineered_result2["stage"] = "engineered_modeling"
```

```

all_results = pd.concat([direct_result2, engineered_result2], axis=0)
all_results = all_results.sort_values("RMSE")
all_results.to_csv(summary_dir / "all_model_comparison.csv",
    ↪ encoding="utf-8-sig")

print("\n" + "=" * 80)
print(" 全部流程完成: 模型对比结果")
print("=" * 80)
print(all_results)

best_model_name = all_results.index[0]
print("\n 最优模型: ", best_model_name)
print(" 最优模型指标: ")
print(all_results.iloc[0])
print(f"\n 全部结果已保存到: {out_dir.resolve()}")

print("\n 建模注意事项: ")
print("1. casual 和 registered 已删除, 因为它们相加就是 cnt, 属于目标泄漏。
    ↪ ")
print("2. 本脚本采用时间顺序切分训练集/测试集, 避免用未来数据预测过去。")
print("3. 重新建模阶段包含: 特征编码、离散化、新特征构造、异常值处理、标准化。
    ↪ ")
print("4. 如果课程要求随机划分, 可把 time_order_train_test_split 改成
    ↪ train_test_split。")

if __name__ == "__main__":
    main()

```

关键运行结果汇总

报告 2 中记录的主要运行结果如下:

数据规模:

17379 行, 18 列; 所有字段缺失值数量均为 0。

直接建模:

Direct_RandomForest RMSE=69.9025, MAE=45.0211, R2=0.8995,
 ↪ NRMSE=0.0716

```
Direct_Ridge_logTarget      RMSE=203.0361, MAE=139.8291, R2=0.1520,  
↪  NRMSE=0.2080
```

特征工程后建模:

```
FE_RandomForest_logTarget  RMSE=77.2029, MAE=50.6933, R2=0.8774, NRMSE=0.0791
```

```
FE_Ridge_logTarget         RMSE=97.2286, MAE=61.1494, R2=0.8055, NRMSE=0.0996
```

重要特征:

```
is_night=0.4919, hr_sin=0.1916, temp=0.0534,
```

```
is_peak_hour=0.0487, atemp=0.0362, hr_cos=0.0261。
```